

Systemarchitektur & Spezifikation: Tischtennis-Vereinsmanager

Dieses Dokument dient als technische Vorgabe für GitHub Copilot / Cursor zur Implementierung des Projekts. Es beschreibt den vereinheitlichten Technologie-Stack, die Datenfluss-Architektur sowie die Vorbereitungsmaßnahmen.

1. Architektur-Diagramm (Mermaid-Spezifikation)

Das folgende Diagramm beschreibt die bereinigte Struktur mit geteiltem Frontend, zentralem API-Gateway, logisch getrennten Backend-Services und einer konsolidierten PostgreSQL-Datenbank.

```
graph TD
    %% Frontend Layer
    subgraph Frontend [Frontend: React + TypeScript]
        WA[Web App: React + Vite]
        DA[Desktop App: Tauri + React]
    end

    %% Gateway Layer
    AGW[API Gateway: ASP.NET Core]

    %% Backend Layer
    subgraph Backend [Backend Services: .NET 8 / C#]
        AUTH[Auth Service]
        SAIS[Saison Service]
        TURN[Turnier Service]
        SPIEL[Spieler Service]
    end

    %% Database Layer
    subgraph Database [Datenschicht]
        DB[(PostgreSQL Datenbank)]
    end

    %% Connections
    Frontend -->|HTTPS / REST| AGW
    AGW -->|JWT Auth & Routing| Backend
    SAIS -->|Interner Aufruf / Referenz| SPIEL
    TURN -->|Interner Aufruf / Referenz| SPIEL
    Backend -->|EF Core ORM / Einteiliges DbContext| DB

    style Frontend fill:#0d5c3a,stroke:#fff,stroke-width:2px,color:#fff
    style Backend fill:#4a47a3,stroke:#fff,stroke-width:2px,color:#fff
    style Database fill:#4a5568,stroke:#fff,stroke-width:2px,color:#fff
    style AGW fill:#2b6cb0,stroke:#fff,stroke-width:2px,color:#fff
```

2. Technischer Stack (Tech-Stack)

Schicht	Technologie / Framework	Details & Einsatzzweck
Frontend	React 18+, TypeScript, Vite	Single Page Application (SPA), Shared Codebase für Web und Desktop. Schnelle Builds via Vite.
Desktop Wrapper	Tauri (v2)	Nativer, leichtgewichtiger Wrapper für Windows, Mac und Linux. Nutzt das React-Build-Artefakt.
API Gateway	ASP.NET Core (YARP oder Custom)	Zentraler Einstiegspunkt. Übernimmt globales Rate Limiting, CORS-Richtlinien und die initiale Validierung von JWTs.
Backend Services	.NET 8 / C#	Logisch separierte Services (Projekte innerhalb einer Solution). Modularer Monolith-Ansatz für einfache Wartung.
Datenzugriff (ORM)	Entity Framework Core 8	Code-First Ansatz. Ein zentrales DbContext für die konsolidierte PostgreSQL-Datenbank zur Wahrung von Fremdschlüssel-Beziehungen.
Datenbank	PostgreSQL 15+	Zentrale relationale SQL-Datenbank. Erlaubt externen, authentifizierten Zugriff. Integrierte Tabellen ohne künstliche Datensilos.

3. Definition der Backend-Services

- **Auth Service:** Zuständig für Login, Benutzerverwaltung und das Ausstellen von zustandslosen JSON Web Tokens (JWT). Verwaltet Rollen wie *Admin*, *Trainer* und *Spieler*.
- **Spieler Service:** Das "Single Source of Truth" für Spielerdaten. Jedes Vereinsmitglied existiert genau einmal. Verwaltet Profile und historische Statistiken.
- **Saison Service:** Nutzung der Spieler-IDs aus dem Spieler-Service zur Erstellung von Spielplänen, Spieltagen und Gruppen für die reguläre Saison sowie Pflege der Ligaspiele.
- **Turnier Service:** Durchführung von Turnieren (K.o.-System, Schweizer System oder Gruppenphasen). Erstellt Brackets und erfasst Live-Ergebnisse. Referenziert Spieler-IDs direkt aus den konsolidierten Tabellen.

4. Entwickler-Anweisungen für den Copilot

Wichtige Programmier-Richtlinien:

1. **Keine Datenredundanz bei Spielern:** Spieler-Entitäten dürfen nicht in den Turnier- oder Saisontabellen dupliziert werden. Nutze relationale Fremdschlüssel über das gemeinsame EF Core Model.

2. **Zustandslose Authentifizierung:** JWTs werden im API Gateway validiert. Die Backend-Services vertrauen der Validierung des Gateways oder nutzen dieselbe Middleware. Token werden nicht in der DB gespeichert.
3. **EF Core Migrations:** Verwende ein dediziertes Migrations-Projekt oder binde die Migrationen zentral in das Hauptprojekt ein, um Schema-Konflikte zu vermeiden.

5. Checkliste: Was du vorbereiten solltest (Vorarbeiten)

Damit der Copilot ohne Unterbrechung und Fehler Code generieren kann, solltest du folgende Infrastruktur und Projektstrukturen im Vorfeld bereitstellen:

A. Infrastruktur & Datenbank (Dein wichtigster Punkt)

- **PostgreSQL Instanz starten:** Setze lokal oder auf einem Server (z.B. via Docker) eine PostgreSQL-Datenbank auf.
- **Verbindungszeichenfolge (Connection String):** Halte die Zugangsdaten bereit. Copilot benötigt diese für die `appsettings.json`, um EF Core Migrationen auszuführen. Die leere Datenbank sollte physisch existieren (z.B. Name: `tt_vereinsmanager`).

B. Projekt-Grundstruktur (Die Solution)

- **.NET Solution anlegen:** Erstelle eine leere .NET Solution (`.sln`) und lege die Ordnerstruktur für die Services an:

```
src/
├── ApiGateway/
├── Services/
│   ├── AuthService/
│   ├── SpielerService/
│   ├── SaisonService/
│   └── TurnierService/
└── Data/ (für das gemeinsame DbContext & Migrationen)
```

- **Frontend-Initialisierung:** Erstelle den React-Workspace mittels Vite im Ordner `frontend/`:

```
npm create vite@latest frontend -- --template react-ts
```

C. Umgebungsvariablen & Zertifikate

- **JWT Secret Key:** Generiere einen sicheren, zufälligen String (mindestens 256-bit) für die JWT-Signierung und lege ihn als Umgebungsvariable oder im User-Secrets-Speicher von .NET ab.